Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices basiques

Le but de cette séance est de découvrir et de mettre en pratique un paradigme de programmation largement utilisé : la programmation orientée objet (POO). Ce paradigme repose sur la création d'objets qui regroupent des données et des comportements, et qui peuvent interagir entre eux. Au fil des exercices, vous apprendrez à manipuler des objets et à maîtriser les notions fondamentales de classe, d'attribut, de méthode et d'encapsulation. À l'issue de cette séance, vous serez capables de concevoir des programmes mieux structurés et plus faciles à maintenir.

Les exercices de cette séance seront réalisés en Java et en Python.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Itération en Java

Question 1: (🕓 *5 minutes*)

> **Conseil**
>
> La méthode `String.valueOf` permet de convertir un nombre en chaîne de caractères.

Qu'affiche le programme Java suivant (par souci de concision, nous omettons la déclaration de la classe et de la méthode `main`) :

```
1 String result = "";
2 String x = "5";
3 int j = 0;
4 for (j=Integer.parseInt(x); j !=0; j = j/2){
5     result = String.valueOf(j%2) + result;
6 }
7 System.out.println(result + j);
```

Choisissez parmi les éléments suivants :

- `5210`
- `1110`
- `1010`
- `1011`
- `java: bad operand types for binary operator '+'..`

2 Récursion en Java et Python

Question 2: (🕓 *10 minutes*) [Java et Python] **isPalindrome()**

> **Conseil**
>
> En Java, pour obtenir le caractère à l'indice i d'une chaîne de caractères, `myString`, utilisez `myString.charAt(i)`. En Python, utilisez `myString[i]`.
>
> Une sous-chaîne est obtenue en Java avec `myString.substring(i, j)` où i est l'indice du premier caractère et $j-1$ est l'indice du dernier caractère. En Python, utilisez `myString[i:j]`.

Un palindrome est une chaîne de caractères qui se lit de la même manière à l'envers. Plus précisément, soit f une fonction qui renvoie `True` si `a` (de longueur n caractères) est un palindrome :

$$f(a) = f(a[1:n-1]) \wedge (a[0] == a[n-1]) \tag{1}$$

Ici, $a[1 : n-1]$ est la sous-chaîne de a qui comprend tous les caractères à l'exception du premier caractère (le premier caractère étant à l'indice 0) et du dernier caractère (le dernier caractère étant à l'indice $n-1$).

Conseil

Revoir la question 16 du TP3 pour voir les règles concernant la notation des bornes d'une sous-chaîne de caractères.

On dit donc que a est un palindrome si le premier caractère et le dernier caractère sont égaux **et** la sous-chaîne entre les deux est un palindrome.

Le cas de base (une chaîne a d'un seul caractère) est :

$$f(a) = \texttt{True} \text{ si } n = 1$$

et le cas de base d'une chaîne vide : $f(\texttt{""}) = \texttt{True}$.

Écrivez la méthode Java **`isPalindrome(String arg)`** et la fonction Python **`isPalindrome(arg)`** qui renvoient `True` si `arg` est un palindrome.

3 Création de votre première classe en Java (40 minutes)

Le but de cette première partie est de créer votre propre classe en Java. Cette classe sera une classe nommée `Dog()` représentant un chien. Elle aura plusieurs attributs et méthodes que vous implémenterez au fur et à mesure.

Question 3: (*10 minutes*) Création de classe et encapsulation
Commencez par créer une nouvelle classe `Dog` dans votre projet. Ensuite, créez les attributs suivants :

1. Un attribut `public` String nommé `name`
2. Un attribut `private` List nommé `tricks`
3. Un attribut `private` String nommé `race`
4. Un attribut `private` int nommé `age`
5. Un attribut `private` int nommé `mood` initialisé à 5 (correspondant à l'humeur du chien)
6. Un attribut de classe (`static`) `private` int nommé `nb_chiens`

Créez une méthode publique du même nom que la classe (`Dog`). Cette méthode est appelée le `constructeur`, elle va servir à initialiser les différentes instances de notre classe. Un `constructeur` en `Java` aura le même nom que la classe, et le `constructeur` en `Python` sera défini par la méthode `__init__`. Cette méthode prendra en argument les éléments suivants qui seront utilisés pour initialiser les attributs de notre instance :

1. Une chaîne de caractères `name`,
2. Une liste `tricks`,
3. Une chaîne de caractères `race`,
4. Un entier `age`.

Pour finir, cette méthode doit incrémenter l'attribut de classe `nb_chiens` qui va garder en mémoire le nombre d'instances créées.

Conseil

Pour cet exercice, n'oubliez pas de préciser si vos attributs sont `public` ou `private`.
Le mot `static` correspond à un élément de classe (attribut ou méthode), cet élément pourra ensuite être appelé via la classe directement.
Pour attribuer des valeurs à vos attributs d'instance, utilisez le mot-clé `this.attribut`.
Pour accéder aux attributs de classe, utilisez `nom_classe.nom_attribut`.

Question 4: (*10 minutes*) Getters et setters
Il faut maintenant créer des méthodes de type `getter` et `setter` afin d'interagir avec les attributs `private` des instances de la classe. Les `getters` renverront les attributs souhaités tandis que les `setters` les modifieront.
Les `setters` sont souvent utilisés pour modifier la valeur d'attributs privés et ne renvoient rien.

Conseil

Exemple de `getters` et de `setters` :

```
1 public String getName() { // Exemple de getter
2     return name;
3 }
4
5 public void setName(String name) { // Exemple de setter
6     this.name = name;
7 }
```

Vous pouvez directement accéder à des attributs publics en utilisant `nom_instance.attribut` à l'intérieur ou à l'extérieur de la classe.

Créez les méthodes suivantes :

- `getTricks()`
- `getRace()`
- `getAge()`
- `getMood()`
- `setTricks()`
- `setRace()`
- `setAge()`
- `setMood()`

Créez également une méthode de classe permettant de retourner le nombre de `Dog` instanciés (un `getter`).

Conseil

Intellij vous permet de générer automatiquement certaines méthodes telles que les getters et setters. Vous pouvez consulter le lien suivant pour plus d'informations : `https://www.jetbrains.com/help/idea/generating-code.html#generate-delegation-methods`.
Toutefois, pour cet exercice, nous vous encourageons à le faire manuellement.

Question 5: (*5 minutes*) Manipulation d'attributs - Listes
Créez une méthode publique nommée `addTrick(String trick)` qui prend en entrée une chaîne de caractères et l'ajoute à la liste `tricks`.

Conseil

La liste `tricks` est une liste comme les autres. Si vous voulez la modifier, vous aurez besoin de passer par une `LinkedList` temporaire.

Question 6: (*5 minutes*) Manipulation d'attributs - `setter`
Créez deux méthodes permettant de modifier l'attribut `mood` de l'objet `Dog`. La méthode `leash()` décrémentera `mood` de 1 et `eat()` l'incrémentera de 3.

Question 7: (*5 minutes*) Manipulation d'attributs d'une autre instance
Créez une méthode nommée `getOldest (Dog other)` qui prend comme argument un élément de type `Dog`, puis retourne le nom et l'âge du chien le plus âgé au format suivant : "`nomChien` est le chien le plus âgé : il a `ageChien` ans".

> **Conseil**
>
> L'élément `Dog` que vous passez en argument est un objet de type `Dog`, vous pouvez donc lui appliquer les méthodes que vous avez créées tout à l'heure. Faites attention à la façon d'accéder aux différents attributs de votre deuxième chien (pour rappel, les attributs privés ne sont accessibles qu'à travers des `getters` que vous aurez préalablement définis).

Question 8: (*5 minutes*) Redéfinition de méthodes
Créez une méthode `toString()` de type `public` qui retourne une chaîne de caractères contenant toutes les informations d'une instance de `Dog`. Ainsi, dans votre `main`, en faisant `System.out.println(...)` sur une instance de `Dog`, vous obtiendrez un texte au format suivant : "`nomChien` a `ageChien` ans, est un `raceChien` et a une humeur de `moodChien`. Il sait faire les tours suivants : `tricksChien`".

> **Conseil**
>
> La méthode `toString()` permet d'obtenir une représentation textuelle d'un objet. Elle est définie par défaut pour tout objet. Pour la redéfinir, on utilise le mot-clé `@Override` avant la méthode.

Pour contrôler que vos méthodes et attributs ont été implémentés correctement, vous pouvez essayer le code suivant à l'intérieur de votre méthode `main` :

```
1  public class Question8 {
2      public static void main(String[] args) {
3          Dog lola = new Dog("Lola",List.of("rollover"),"Bouvier",10);
4          Dog tobi = new Dog("Tobi",List.of("rollover","do a
       barrel"),"Doggo",17);
5          System.out.println(lola.getAge());
6          System.out.println(lola.getMood());
7          System.out.println(lola.getRace());
8          System.out.println(lola.name);
9          System.out.println(lola.getTricks());
10         lola.setAge(13);
11         lola.setMood(8);
12         lola.setRace("Bouvier");
13         lola.name = "lola";
14         lola.setTricks(List.of("rollover","do a barrel"));
15         lola.eat();
16         lola.leash();
17         lola.addTrick("sit");
18         System.out.println(Dog.getNbChiens());
19         System.out.println(lola.getOldest(tobi));
20         System.out.println(lola);
21     }
22 }
```

Vous devriez obtenir ce résultat :

```
1  10
2  5
3  Bouvier
4  Loola
5  [rollover]
```

```
2
Tobi est le chien le plus agé avec 17 ans
Lola a 13 ans, est un Bouvier et a une humeur de 10. Il sait faire les
    tours suivants : [rollover, do a barrel, sit]
```

4 Notions de POO en Python (15 minutes)

Dans cette section, nous créerons pas à pas une classe `Point` contenant des attributs et des méthodes utiles. Dans votre IDE, créez un nouveau projet Python (Fichier > Nouveau > Projet). Dans un dossier de votre choix, créez un fichier `question7.py`.

Question 9: (*15 minutes*) Classe `Point`

— Créez une classe `Point` et un constructeur par défaut contenant deux paramètres (`x` et `y`).

> **Conseil**
>
> Pour rappel, un constructeur est une fonction `__init()__` que vous redéfinirez dans votre classe.

— Définissez deux attributs privés pour votre classe `Point`. Ces attributs seront les coordonnées x et y de vos points. Par défaut, assignez-leur les valeurs données dans le constructeur.

> **Conseil**
>
> À l'intérieur d'une classe, utilisez le mot-clé `self` pour accéder aux méthodes et attributs de l'instance que vous manipulez. En Python, pour spécifier qu'un attribut est privé, ajoutez deux traits de soulignement au nom de l'attribut (exemple : `__score=0`).

— Définissez des getters et des setters.

> **Conseil**
>
> En Python, le mot-clé `self` est l'équivalent de `this` utilisé en Java.

— Définissez une méthode `distance` qui prend en entrée l'instance du `Point` (`self`) et un autre `Point p2`. Cette méthode `distance` retournera la distance euclidienne entre le point `self` et `p2`.

> **Conseil**
>
> Pour rappel, la distance euclidienne entre deux points est définie par la formule $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Utilisez la fonction `sqrt()` de la bibliothèque `math` pour calculer la racine carrée. Pensez à importer la bibliothèque `math`.

— Définissez une méthode `milieu` qui prendra en entrée `self` et `p2` et qui retournera un objet `Point` situé entre `self` et `p2`.

> **Conseil**
>
> Pour trouver les coordonnées d'un point M(x_M, y_M) situé au milieu du segment défini par des points A(x_A, y_A) et B(x_B, y_B), utilisez les formules suivantes : $x_M = \frac{x_1+x_2}{2}$ et $y_M = \frac{y_1+y_2}{2}$

— Redéfinissez une méthode `__str__()` dans la classe `Point` qui retournera une chaîne de caractères contenant les coordonnées (x, y) d'un point. Ainsi, lorsqu'on fera un `print` d'une instance de la classe `Point`, le message qui s'affichera sera le suivant : *Les coordonnées du Point sont : x = "remplacez par la valeur de x" et y = "remplacez par la valeur de y"*

5 Notions d'héritage - Java (30 minutes)

Le but de cette partie est de mettre en pratique les notions liées à l'héritage. Nous allons créer une classe `Livre()` qui représentera notre classe-mère. Nous allons également créer deux classes filles, `LivreAudio()` et `LivreIllustre()`. Les classes filles hériteront des attributs et méthodes de la classe-mère.

Question 10: (*20 minutes*) `Création des différentes classes`
Créez la classe-mère `Livre` avec les caractéristiques suivantes :

- un attribut `privé String` nommé `titre`,
- un attribut `privé String` nommé `auteur`,
- un attribut `privé int` nommé `annee`,
- un attribut `privé int` nommé `note` (initialisé à `-1`),
- le `constructeur` de la classe qui prendra les trois premiers arguments cités ci-dessus,
- une méthode `setNote()` qui permet de définir l'attribut `note`,
- une méthode `getNote()` qui permet de retourner l'attribut `note`,
- une méthode `toString()` qui retournera le titre, l'auteur, l'année et la note d'un ouvrage (réécrire cette méthode permettra d'afficher un objet `Livre` en utilisant `System.out.println()`).

Attention, si la `note` n'a pas été modifiée et qu'elle vaut toujours `-1`, affichez "Note : pas encore attribuée" au lieu de "Note : `note`" via la méthode `toString()`.
Créez les classes filles avec les caractéristiques suivantes :
`class LivreAudio extends Livre`

- un attribut `privé String` nommé `narrateur`

`class LivreIllustre extends Livre`

- un attribut `privé String` nommé `illustrateur`

Voici le squelette du programme à compléter :

```
1  class Livre {
2  }
3  class LivreAudio extends Livre {
4  }
5  class LivreIllustre extends Livre {
6  }
```

Conseil

- En Java, lors de la déclaration d'une classe, le mot-clé `extends` permet d'indiquer qu'il s'agit d'une classe fille de la classe indiquée. Le mot-clé `super` permet à la sous-classe d'hériter d'éléments de la classe-mère. `super` peut être utilisé dans le constructeur de la classe-fille selon l'exemple suivant : `super(attribut_mère_1, attribut_mère_2, attribut_mère_3, etc.);`. Ainsi, il n'est pas nécessaire de redéfinir tous les attributs d'une classe fille !
- L'instruction `super` doit toujours être la première instruction dans le constructeur d'une classe-fille.
- Vous pouvez vous servir de `'\n'` dans une chaîne de caractères pour effectuer un retour à la ligne lors de l'affichage d'une chaîne de caractères.

Question 11: (*10 minutes*) `Méthode et héritage`
Maintenant que vous avez créé la classe-mère et les classes filles correspondantes, vous pouvez créer un objet `Livre` à l'aide du constructeur de la classe `LivreAudio` (et des arguments donnés lors de la création de l'objet). En instanciant l'objet, vous pourriez utiliser les valeurs suivantes :
titre : "Hamlet", auteur : "Shakespeare", année : "1609" et narrateur : "William".
Une fois l'objet créé, attribuez-lui une note à l'aide de la méthode `setNote()` définie précédemment.

Finalement, utilisez la méthode `System.out.println()` pour afficher les informations du livre. Redéfinissez la méthode `toString()` de la classe `LivreAudio` afin que la valeur de l'attribut `narrateur` soit affichée. Faites pareil avec la classe `LivreIllustre` et son attribut `Illustrateur`.

Conseil

Attention, vous devez créer un objet `Livre` et non `LivreAudio`. Le mot-clé `super` peut être utilisé dans la redéfinition d'une méthode selon l'exemple suivant : `super.nom_de_la_methode();`. Le mot-clé `super` représente la classe parente, tout comme le mot-clé `this` fait référence à l'instance avec laquelle la méthode est appelée. L'instruction `super` doit toujours être la première instruction dans la redéfinition d'une méthode dans une classe fille.

Lorsque toutes les étapes auront été effectuées, placez le code suivant dans votre `main` et exécutez votre programme :

```
public class Question11 {

    public static void main(String[] args) {
        Livre Livre1 = new Livre_Audio("Hamlet", "Shakespeare",
    1609,"William");
        Livre1.setNote(5);
        Livre Livre2 = new Livre("Les Misérables","Hugo",1862);
        System.out.print(Livre1);
        System.out.print(Livre2);
    }
}
```

Vous devriez obtenir :

```
Création d'un livre
Création d'un livre audio
Création d'un livre
A propos du livre
------------------
Titre : Hamlet
Auteur : Shakespeare
Année : 1609
Note : 5
Narrateur: William
A propos du livre
------------------
Titre : Les Misérables
Auteur : Hugo
Année : 1862
Note : non attribuée
Process finished with exit code 0
```